

Practica 14 (Final): Algoritmo de Metropolis-Hastings

En esta práctica aprenderán a programar el algoritmo de Metropolis-Hastings para simular variables aleatorias.

La idea del algoritmo para simular variables aleatorias con distribución π es la siguiente:

1. Se toma una semilla $X_0 = x_0$

Simulación de un nuevo estado

1. Dado $X_n = x$, se elige el salto potencial a un estado Y_n con probabilidad de salto $q(x, y)$ (un candidato para el siguiente valor de muestra y , dado el valor de muestra anterior x)

$$\text{simulamos } Y_n \sim q(x, y)$$

Calculo de la probabilidad de aceptación

$$\text{setoma } X_{n+1} = \begin{cases} y & \text{con probabilidad } \alpha(x, y) \\ x & \text{con probabilidad } 1 - \alpha(x, y) \end{cases}$$

donde

$$\alpha(x, y) = \min \left\{ 1, \frac{\pi(y)q(y, x)}{\pi(x)q(x, y)} \right\}$$

Observación: $\alpha(x, y) \in [0, 1]$

Observen que $q(x, y)$ un kernel de transición de probabilidad y puede haber diferentes propuestas del mismo.

Una propuesta conveniente de $q(x, y)$ es la siguiente:

$$q(x, y) = f_{U_{xr}}(y) \sim \text{Unif}(x-r, x+r)$$

donde $f_{U_{xr}}$ es la función de densidad de una Uniforme $(x-r, x+r)$ con $r \in \mathbb{R}$.

Esto resulta muy conveniente porque las uniformes son fáciles de simular y además resulta que $q(x, y) = q(y, x)$ ya que:

$$q(x, y) = f_{U_{xr}}(y) = \begin{cases} \frac{1}{2r} & \text{si } |y - x| < r \\ 0 & \text{e.o.c} \end{cases}$$
$$q(y, x) = f_{U_{yr}}(x) = \begin{cases} \frac{1}{2r} & \text{si } |x - y| < r \\ 0 & \text{e.o.c} \end{cases}$$

y así $\alpha(x, y)$ se simplifica a

$$\alpha(x, y) = \min \left(1, \frac{\pi(y)}{\pi(x)} \right)$$

Ejercicios:

En estos ejercicios contruiremos paso a paso el algoritmo de Metropolis-Hastings para generar números aleatorios provenientes de la distribución Beta(a,b).

Es decir $\pi = \text{beta}(a, b)$

Ejercicio 1:

Define una función que reciba como parámetros:

- x
- a
- b

que se encarga de evaluar la función de densidad de la distribución $\text{beta}(a, b)$ en x . Luego, grafica la función de densidad en el intervalo $(0, 1)$ usando los siguientes parámetros y muestra todas las curvas en una sola gráfica.

Parámetros:

- $a=0.5, b=0.5$
- $a=5, b=1$
- $a=1, b=3$
- $a=2, b=2$
- $a=2, b=5$

Recuerda que **la función de densidad de la $\text{beta}(a, b)$** está dada por

$$f(x) = \frac{x^{a-1}(1-x)^{b-1}}{B(a, b)} \text{ si } 0 < x < 1$$

donde **B(a,b)** es la **función Beta**, se define como

$$B(a, b) = \int_0^1 x^{a-1}(1-x)^{b-1} dx$$

Para calcular **función Beta** puedes usar `scipy.special.beta`, la cual ya encontrarás importada en la siguiente línea de código como `beta`

```
import matplotlib.pyplot as plt
from scipy.special import beta
```

```
beta(1, 1)
```

1.0

Ejercicio 2:

Observa que en el cálculo computacional de $\alpha(x, y)$ puede haber un **error** destruya al algoritmo. Este error se debe a que hay valores de x para los cuales $\pi(x)=0$ y entonces $\frac{\pi(y)}{\pi(x)}$ no está definido. Sin embargo podemos corregir ese error pidiendo

$$\alpha(x, y) = \begin{cases} 1 & \text{si } \pi(x) = 0 \\ \min\left\{1, \frac{\pi(y)}{\pi(x)}\right\} & \text{en otro caso} \end{cases}$$

Programa una función que reciba como parámetros a

- x
- y
- π

y que calcule $\alpha(x, y)$ tomando en cuenta el problema mencionado anteriormente.

Ejercicio 3:

En este ejercicio deberán programar una función que se encargue de los pasos 2) y 3) del algoritmo de Metropolis-Hastings. Para ello su función deberá recibir

- el valor x del estado X_n
- el tamaño del paso de la distribución uniforme, r
- la función de distribución π

como parámetros y deberá hacer lo siguiente:

Simulación de un nuevo estado

1. Generar y usando $q(x, y) = f_{U_r}(y)$. Para esto simplemente deben simular una variable aleatoria $\text{Uniforme}(x-r, x+r)$ y asignarle ese valor a y .

Calcular la probabilidad de aceptación

1. Calcular el valor de $\alpha(x, y)$ usando la función del Ejercicio anterior.

Asignar X_{n+1}

1. Simular una variable aleatoria U con distribución $\text{Uniforme}(0, 1)$.

$$set\ o\ m\ a\ X_{n+1} = \begin{cases} y & \text{si } U \leq \alpha(x, y) \\ x & \text{si } U > \alpha(x, y) \end{cases}$$

2. Regresar X_{n+1}

Ejercicio 4:

Usando la función del ejercicio anterior ya es muy fácil programar el algoritmo de Metropolis-Hastings y en este ejercicio se encargarán justamente de ello.

Con esto su función debe hacer lo siguiente:

1. Inicial la lista de muestras con X_0 .
2. Para i el rango de $(1, N)$:
 Mandar a llamar a la función del Ejercicio 3
 con $x=X_{i-1}$, π y r
 guardar su valor en la lista de muestras.
3. Regresar la lista de muestras

Deberán definir una función que reciba como parámetros:

- El número de iteraciones del algoritmo, N , el cual equivale al número de muestras que van a generar de la variable Beta(a,b)
- La condición inicial del algoritmo, X_0
- La función de densidad de la variable a simular, π , (en este caso la de la Beta(a,b))
- r el tamaño del paso de la distribución Uniforme($x-r, x+r$).

Ejercicio 5:

Prueba la función del ejercicio anterior con 1000 iteraciones, condición inicial $X_0=3$, la función de densidad de la Beta(2,5) y $r=1$.

Grafica la lista de valores obtenidos contra el número de iteraciones y comenta que observas y qué le cambiarías al algoritmo para mejorar su efectividad

Ejercicio 6:

Prueba tu función del algoritmo de Metropolis-Hastings usando 10,000 iteraciones y un periodo de quemado de 1000 para tres de las densidades Beta del Ejercicio 1. En cada caso grafica el histograma de las muestras junto con la densidad las variables simuladas y varía r para ver si cambian los resultados.